Mock A - Marking Scheme and Walkthrough

Campus cafe orders: pandas + matplotlib + SQLite + PyTorch. 40 marks. Model solution: `MockA_solution.py` *(tested: Python 3.14, pandas 3.1, torch 2.14). Data:* `cafe_orders.csv` *(169 data rows).*

1. Marks overview and self-marking

Sub-task	Marks	Full marks need
1a load	2	`pd.read_csv`, DataFrame kept in a variable
1a missing values	2	no NaN left; fill values sensible
1a duplicates	1	`drop_duplicates()` (169 -> 164 rows)
1a dates	2	`pd.to_datetime` on OrderTime; correct `format=`
1a Total fix	2	Total recomputed as Quantity x Price
1b bar chart	2	one bar per item, title + both axis labels
1b line plot	2	2024 only, months 1-12 on x, title + labels
2a database	2	`sqlite3.connect("CafeDB.db")`
2a table	3	`CREATE TABLE Orders` with 5 sensible columns/types
2a insert	4	rows inserted, parameters `?`, `commit()`, no duplicates
2b total 2024	3	correct SUM filtered to 2024, printed
2b item summary	3	GROUP BY + SUM + ORDER BY ... DESC
3a	3	class `CafeNN` (6-8-4-1), BCELoss, Adam lr 0.01
3b	2	`torch.randn(200, 6)`, target 200 x 1 from the rule
3c	3	30-epoch loop: zero_grad, forward, loss, backward, step, record
3d	2	plot of losses, title + labels
3e	2	predictions > 0.5, accuracy printed as percentage

Style marks: the real exam also rewards clear names, comments, tidy layout. If your code is readable and commented, you are safe.

How to self-mark: run your file, compare your printed numbers with section 2, then walk down the table. Give yourself a mark only when you can point to the line of code that earns it. Half marks for "right idea, small bug" (e.g. forgot `commit()`).

2. Expected results (verified by running, and by an independent

plain-Python calculation using the `csv` and `decimal` modules)

```
Rows, columns: (169, 5)
Missing values BEFORE: Item 3, Quantity 2, Price 2, Total 1
Missing values AFTER:  all 0
Duplicate rows removed: 5
Rows where Total was wrong: 3
Rows now in Orders table: 164            (2023: 20, 2024: 120, 2025: 24)
Total sales in 2024: 871.40
Item sales summary for 2024:
  Mocha 56 | Latte 52 | Espresso 41 | Muffin 36 | Flat White 34
  Cappuccino 29 | Tea 28 | Americano 23 | Unknown 7
```

Bar chart values (quantity, all years, after cleaning): Latte 73, Mocha 68, Espresso 53, Muffin 52, Cappuccino 51, Tea 42, Flat White 38, Americano 36, Unknown 7 (nine bars).

Line plot values (2024 total sales per month, GBP): Jan 74.60, Feb 57.10, Mar 61.20, Apr 70.60, May 72.20, Jun 93.00, Jul 68.90, Aug 85.50, Sep 74.50, Oct 67.30, Nov 72.00, Dec 74.50 (peak June, dip February). (They add up to 871.40, matching query 1.)

Neural network (seed 42, CPU; the last decimal can differ by machine):

```
Epoch  1/30  loss = 0.7178      Epoch 20/30  loss = 0.5309
Epoch 10/30  loss = 0.6572      Epoch 30/30  loss = 0.3474
Training accuracy: 92.5%
```

The loss must fall smoothly from about 0.72 to about 0.35, and accuracy should be well above 50% (the two classes are exactly balanced, so 50% is "guessing"). Exact decimals do not matter for marks; the trend does.

3. Planted traps in the data and how the solution handles them

Trap in `cafe_orders.csv`	Where	Handling in the solution
Blank `Item` (3 cells; 2 distinct orders)	2024/08/09 15:25; 2024/08/15 15:40	`fillna("Unknown")`
Blank `Quantity` (2)	2023/12/13 09:15; 2024/12/10 15:40	Quantity = Total / Price (=4 both), fallback 0
Blank `Price` (2)	2024/01/28 09:10; 2025/03/09 10:30	Price = Total / Quantity (1.80; 3.30)
Blank `Total` (1)	2024/01/22 12:20	Total = Quantity x Price (9.60)
Wrong `Total` (3)	2024/08/08 08:15 (3.60 not 10.80); 2024/10/07 15:35 (4.60 not 3.60); 2024/12/28 15:10 (3.10 not 9.30)	recompute Total for every row, `round(2)`
Exact duplicate rows (5 pairs)	all in 2024	`drop_duplicates()` before charts; UNIQUE in the table
Date format `2024/03/05 09:15`	all rows	`to_datetime(..., format="%Y/%m/%d %H:%M")`
Three years in the file	2023, 2024, 2025	filter `dt.year == 2024` for line plot and queries
NaN makes Quantity a float column	Quantity	`.astype(int)` after filling

Other pitfalls worth knowing (all checked on the test machine):

- **Date text with slashes in SQLite:** `strftime('%Y', '2024/03/05 09:15')` returns NULL, so the 2024 query would print `None`. Store `2024-03-05 09:15:00` (hyphens) via `dt.strftime("%Y-%m-%d %H:%M:%S")`.
- **Passing a pandas `Timestamp` to `execute`** raises `sqlite3.ProgrammingError: Error binding parameter ... 'Timestamp' is not supported` on this Python (3.14). Convert to text first.
- **Passing numpy numbers** (`row["Quantity"]` is `numpy.int64`) stores a BLOB (bytes) instead of a number, silently. Wrap in `int()`/`float()`.
- **Alternative fills are acceptable for marks** (blank Quantity = 0 or the median, blank Item = "Unknown"/"Missing"). They give different totals: blank Quantity = 0 makes the 2024 total 864.20 instead of 871.40. A marker checks the *rule* is sensible, not the exact figure.
- **Forgetting `drop_duplicates()` before the charts** inflates the 2024 total to 900.00 in the chart/pandas numbers while the database (thanks to UNIQUE) correctly says 871.40. Mismatch = a clue you missed it.
- `INSERT OR IGNORE` **with no UNIQUE constraint** ignores nothing; the second run doubles the table to 328 rows.

4. Walkthrough of the key lines

Task 1a - cleaning (analogy: a spreadsheet you tidy with a pencil)

```
df = pd.read_csv("cafe_orders.csv")        # file -> table in memory
df["Item"] = df["Item"].fillna("Unknown")  # fill blank text cells
df["Quantity"] = df["Quantity"].fillna((df["Total"] / df["Price"]).round())
df["Quantity"] = df["Quantity"].fillna(0).astype(int)
```

Plain English: where Item is blank, write "Unknown". Where Quantity is blank we can *work it out*: Total / Price (like finding how many cups were bought from the bill). If we cannot, use 0. `astype(int)` turns 4.0 back into 4, because NaN had forced the whole column to decimals.

```
df = df.drop_duplicates()                  # remove photocopied rows
df["OrderTime"] = pd.to_datetime(df["OrderTime"], format="%Y/%m/%d %H:%M")
correct_total = (df["Quantity"] * df["Price"]).round(2)
df["Total"] = correct_total                # overwrite every Total
```

`format=` tells pandas exactly how to read the text, like telling a postman which line of an address is the street. Recomputing all totals fixes the 3 wrong rows (and the blank) in one step. `round(2)` removes floating-point dust such as 9.600000000000001.

Task 1b - charts (analogy: a pivot table, then a picture)

```
qty_per_item = df.groupby("Item")["Quantity"].sum()   # 1 number per item
plt.bar(qty_per_item.index, qty_per_item.values)      # one bar per item
df_2024 = df[df["OrderTime"].dt.year == 2024]         # keep 2024 rows
monthly = df_2024.groupby(df_2024["OrderTime"].dt.month)["Total"].sum()
plt.plot(monthly.index, monthly.values, marker="o")   # month vs sales
```

`groupby` = sort receipts into piles by item, then add up each pile. `.dt.month` pulls the month number (1-12) out of each datetime.

Task 2 - SQLite (analogy: filing cabinet, clerk, bouncer)

```
conn = sqlite3.connect("CafeDB.db")   # open (or create) the cabinet
cursor = conn.cursor()                # the clerk who does the work
cursor.execute("""CREATE TABLE IF NOT EXISTS Orders (... ,
    UNIQUE (OrderTime, Item, Quantity, Price, Total))""")
cursor.execute("INSERT OR IGNORE INTO Orders ... VALUES (?, ?, ?, ?, ?)",
               (...))
conn.commit()                         # press Save on the form
```

`UNIQUE` is the bouncer: an identical row is refused. `OR IGNORE` says "refuse quietly instead of crashing". `?` placeholders keep the data separate from the SQL text (safe and avoids quote problems). Re-running the program therefore never adds rows: 164 every time.

```
SELECT ROUND(SUM(Total), 2) FROM Orders WHERE strftime('%Y', OrderTime)='2024'
SELECT Item, SUM(Quantity) AS TotalQty FROM Orders
WHERE strftime('%Y', OrderTime) = '2024'
GROUP BY Item ORDER BY TotalQty DESC, Item
```

`WHERE` first filters rows (only 2024), `GROUP BY` makes one pile per item, `SUM` adds each pile, `ORDER BY ... DESC` puts the biggest first. `, Item` is a tie-breaker so equal quantities print in a fixed order.

Task 3 - PyTorch (analogy: a hiker feeling their way down a hill)

```
model = CafeNN()                                  # the network
criterion = nn.BCELoss()                          # "how wrong?" score
optimizer = optim.Adam(model.parameters(), lr=0.01)  # the hiker
for epoch in range(30):
    optimizer.zero_grad()            # wipe last step's whiteboard notes
    output = model(data)             # guess the labels
```

```
    loss = criterion(output, target) # score the guesses
    loss.backward()                  # work out which way is downhill
    optimizer.step()                 # take a small step that way
    losses.append(loss.item())       # note the height (loss) as a number
```

Sigmoid squashes the output into 0..1 (a probability) which is what `BCELoss` expects for yes/no labels. Loss = "height of the hill"; the hiker steps downhill so it falls. In 3e `torch.no_grad()` means "just look, do not learn", and `> 0.5` turns a probability into a 0/1 answer.

5. Viva-style follow-up questions

1. **Why did you pass `format=` to `to_datetime`?** Because the text is `year/month/day hour:minute`; being explicit stops pandas guessing (a guess can swap day and month on a format like 05/03/2024) and is faster. `dayfirst=True` would be wrong here.
2. **Why do you need both `INSERT OR IGNORE` and `UNIQUE`?** `UNIQUE` defines what "duplicate" means and rejects it; `OR IGNORE` turns the rejection into a silent skip instead of an `IntegrityError`. Alone, `OR IGNORE` has no rule to break.
3. **Why convert with `int()` / `float()` before inserting?** pandas hands out numpy numbers; sqlite3 may store them as raw bytes (BLOB), which breaks `SUM`. Python natives are stored as proper numbers.
4. **Why store dates with hyphens, and why not use the pandas Timestamp directly?** SQLite date functions only understand ISO text `YYYY-MM-DD HH:MM:SS`; slashes give NULL. A Timestamp object cannot be bound as a parameter in current Python.
5. **Why BCELoss with a Sigmoid output, not MSELoss?** The task is yes/no classification; sigmoid gives a probability and BCE is the matching loss (it punishes confident wrong answers heavily). MSE also works but trains worse on probabilities.
6. **What does `optimizer.zero_grad()` do and what happens if you omit it?** PyTorch *adds* new gradients to old ones; without clearing, every epoch's step uses a growing pile of stale gradients and training goes wrong.
7. **Is 92.5% training accuracy proof the model is good?** No. It was measured on the same data it learned from. A fair test uses unseen (test) data. With 200 samples and 30 epochs it has only learned a rough version of the rule.
8. **Why recompute Total for all rows instead of only the wrong ones?** It is simpler and safe: for correct rows it changes nothing, and it fixes the blank Totals as well. `round(2)` avoids floating-point noise that would otherwise create "different" values in the UNIQUE check.
9. **What would you change so the query works for any year?** Put the year in a variable and use a placeholder: `WHERE strftime('%Y', OrderTime) = ?` with `(str(year),)`.